User guide

Fault Tree & PFD Calculator – how to build a fault tree, enter reliability data, calculate the system PFDavg and read the results.

Contents

1. What the app does
2. Quick start – your first tree in ten minutes
3. Screen layout
4. Fault tree basics and symbols
5. Building and editing the tree
6. Gate types and voting (MooN)
7. Basic event models and data entry
8. Common-cause failures (β-factor)
9. How the calculation works
10. Reading the results
11. SIF calculator (IEC 61508-6)
12. Worked examples – V-101 overpressure protection; hot DRI conveyor N2 fire protection
13. Saving, files and export
14. Good practice, assumptions and limitations
15. Troubleshooting and FAQ
16. Glossary
17. References

1 · What the app does

A **fault tree** is a top-down, logical model of how an undesired **top event** (for example *"overpressure protection of V-101 fails on demand"*) can be caused by combinations of lower-level failures. This app lets you draw the tree, give each **basic event** its failure data and then calculates, quantitatively:

- the **top-event probability** – for safety systems on demand this is the **average probability of failure on demand (PFDavg)**;
- the corresponding **risk reduction factor** (RRF = 1 / PFDavg) and the **SIL band** the PFDavg falls in (low-demand mode);
- the **minimal cut sets** – the smallest combinations of failures that cause the top event – ranked by contribution;
- **importance measures** showing which events drive the result and where improvement will pay off;
- the **uncertainty band** on the result (Monte Carlo);
- the **top-event frequency** when the tree contains an initiating event (for example a demand rate).

A separate **SIF calculator** applies the simplified equations of IEC 61508-6 Annex B so that a fault tree and the standard formulas can be compared for the same safety instrumented function.

When to use a fault tree rather than the simple formulas: when the system has mixed architectures, shared components (for example one transmitter used by two functions), different proof-test intervals, non-standard voting, human or procedural failures (bypasses, wrong line-up) or several protection layers acting together (SIF + relief valve). The formulas assume a standard, symmetric architecture; the fault tree does not.

2 · Quick start – your first tree in ten minutes

1. **Name the top event.** On the *Fault tree* tab the top gate `G-001` is already selected. Type a precise description in the side panel, e.g. *“Pump P-1 fails to deliver flow on demand”*. A good top event says *what* fails, *where* and *when* (on demand, during the mission…).
2. **Choose the gate type.** Ask: “Does the top event need *all* inputs to fail (AND) or is *any one* enough (OR)?”
3. **Add inputs.** Click + *Gate* for an intermediate event you will develop further, or + *New event* for a basic event that you will give failure data to. The new item is selected so you can describe it straight away.
4. **Enter data.** For a basic event choose the **model** (usually *Periodically proof-tested* for safety equipment) and enter λDU, λDD, the proof-test interval TI and MTTR – all rates **per hour**, times in **hours**.
5. **Read the result.** The banner at the top-left of the drawing shows the top-event PFDavg and its SIL band; every box shows its own time-averaged probability in red. Open the *Results* tab for the detail.
6. **Save.** Work is kept in the browser automatically. Use *File ▸ Save as…* to link a `.fta.json` file (for example on OneDrive) so that every change is also written to that file.

> Tip: *File ▸ Load example* loads a complete tree you can explore – Section 12 explains every number in it.

3 · Screen layout

Area	What it is for
Header	Tree title, save status pill (browser / linked file), *Reconnect file* (appears when the browser needs permission again) and the **File** menu: new, open, save as, backup, import, Excel export, picture export, print report, example, help.
Fault tree	The drawing (left) and the editing side panel (right). Click a box to select it; drag the background to pan; mouse wheel or + / − to zoom; *Fit* to show the whole tree.
Basic events	All basic events in one table – fastest way to enter or check data for many events. Includes events not (yet) used in the tree.
Common cause (β)	Common-cause failure groups and their β values.
Results	PFDavg and comparison values, PFD(t) chart, uncertainty, minimal cut sets and importance measures.
SIF calculator	IEC 61508-6 simplified equations per subsystem, with *Build fault tree from this SIF*.
Settings	Title and description, mission time, number of time points, cut-set truncation limits, Monte Carlo sample size, demand mode.
User guide	This guide.

4 · Fault tree basics and symbols

Each box in the drawing carries a **code** (G-… for gates, BE-… for basic events), a description and – in red – its calculated probability. The symbol below the box shows what kind of element it is:



OR gate
output occurs if *any* input occurs



AND gate
output occurs only if *all* inputs occur



VOTE gate k/n
output occurs if at least k of the n inputs occur



INHIBIT gate
AND with a condition (e.g. “operator absent”)



Basic event
failure with its own data



Undeveloped event
not analysed further (lack of data or scope)



Initiating event
a frequency per year (demand, upset)



House event
switch: TRUE or FALSE (configuration)

Further markers: **·β** after a code = the event belongs to a common-cause group; **↺ shared** = the same event is used in more than one place (it is one variable in the calculation, not two copies); a triangle under a gate = the branch is collapsed.

Key ideas

- **Top event** – the single undesired outcome analysed. Everything in the tree must be a cause of it.
- **Intermediate event** – the output of a gate; it is developed further.
- **Basic event** – the limit of resolution; it has failure data.
- **Cut set** – a set of basic events whose joint occurrence causes the top event. A **minimal** cut set cannot be reduced: remove any one event and the top event no longer occurs. A cut set of **order 1** is a single point of failure.
- **Immediate-cause principle** – develop each gate only by its *immediate*, necessary and sufficient causes; do not jump levels. Keep the “no gate-to-gate” rule: every gate output must be a clearly described event.

5 · Building and editing the tree

Selecting and the side panel

Click any box. For a **gate** the panel shows code, type, description, the list of inputs and the buttons below. For a **basic event** it shows the data form for the chosen model. Text you type is saved as you type; the drawing and results refresh when you leave the field.

Button (gate selected)	Effect
+ Gate	Adds a new intermediate gate as an input and selects it.
+ New event	Creates a new basic event (code BE-xxx) as an input and selects it.
+ Existing event	Adds an event that already exists elsewhere – it becomes a **shared** event. Use this for genuinely common equipment (one transmitter feeding two trip channels, a common power supply, the same operator action).
↑ ↓ →	Reorder inputs (affects the drawing only) or jump to an input.
Collapse / Expand branch	Hides a large branch in the drawing; it is still calculated.
Select parent / Delete branch	Navigate up; delete the gate and all gates below it (basic events stay in the event list).

Button (event selected)	Effect
Make independent copy here	Replaces a shared event at this position by its own copy, so the two places can have different data.
Remove from G-xxx	Removes the event from this gate only; it stays in the event list (Basic events tab) and elsewhere in the tree.

> Shared events are the main reason to use a fault tree tool: the calculation treats a shared event as *one* variable, so a common transmitter or common power supply is not double-counted and its dependency is captured exactly.

6 · Gate types and voting (MooN)

Gate	Logic	Probability (independent inputs)
OR	any input	$1 - \Pi(1 - q_i)$ ≈ Σq_i when all q_i are small
AND	all inputs	$\Pi\ q_i$ (instantaneous; see Section 9 for averaging)
VOTE k/n	at least k of n inputs	e.g. 2/3 with equal q: $3q^2 - 2q^3$
INHIBIT	AND with a condition	as AND

The app always calculates these exactly, including shared events – the formulas above are for orientation only.

Converting a voting architecture to a failure gate

Safety-system architectures are written **MooN** = “M out of N channels must work (vote to trip)”. A fault tree models *failure*, so the failure gate is **(N − M + 1) out of N**:

Architecture	Trips when…	Fails on demand when…	Gate to use
1oo1	the channel trips	the channel fails	event directly / OR
1oo2	either channel trips	both fail	AND (= VOTE 2/2)
2oo2	both trip	either fails	OR (= VOTE 1/2)
2oo3	two of three trip	two of three fail	VOTE 2/3
1oo3	any one trips	all three fail	AND (= VOTE 3/3)

> 2oo2 is *less* safe than 1oo1 for failure on demand (but better against spurious trips) – the OR gate makes this obvious.

7 · Basic event models and data entry

All failure rates are entered **per hour** and all times in **hours**. Useful conversions: 1 year = 8 760 h · 1 FIT = 10^{-9} /h · a rate of 1 per 100 years = 1/(100 × 8 760) = 1.14×10^{-6} /h · MTBF (h) = 1/λ.

Model	Inputs	Unavailability used	Typical use
Periodically proof-tested	λDU, λDD (/h), TI, MTTR (h)	$q(t) = 1 - e^{-\lambda DU\cdot(t \bmod TI)} + \lambda DD\cdot MTTR$ average ≈ λDU·TI/2 + λDD·MTTR	Sensors, logic solvers, valves, relief valves, alarms – any standby equipment that is proof-tested.
Fixed probability	q	q (constant)	Human error probabilities, probability a condition is present, conditional modifiers, generic PFDs from a LOPA table.
Non-repairable over mission	λ (/h); mission time in Settings	$q(t) = 1 - e^{-\lambda t}$, averaged over the mission	Equipment that cannot be repaired during a mission or campaign.
Repairable, revealed	λ (/h), MTTR (h)	$q = \lambda\cdot MTTR / (1 + \lambda\cdot MTTR)$	Running equipment whose failure is immediately evident and is repaired (steady-state unavailability).
Initiating event frequency	f (/yr)	used as a frequency (see Section 9)	Demand rate, upset frequency, loss of utility – turns the result into a frequency.
House event	TRUE / FALSE	1 or 0	Switch branches on or off (e.g. “bypass in place”, “spare pump running”) to compare configurations.

Dangerous undetected vs. detected failures

- **λDU** – dangerous failures found only by the **proof test**. They accumulate between tests, so their contribution grows with TI (sawtooth).

- **λDD** – dangerous failures revealed by **diagnostics** within a short time and repaired within **MTTR**; contribution ≈ λDD × MTTR. (If the diagnostics take the process to a safe state instead, enter λDD = 0.)
- From a data sheet: λDU = λD × (1 − DC), λDD = λD × DC, where DC is the diagnostic coverage.

Error factor (EF)

Used only by the Monte Carlo uncertainty run. EF is the ratio of the 95th percentile to the median of a lognormal distribution: EF = 3 means the true value is believed to lie within about a factor 3 either side of the median (90 % interval). Typical values: well-documented generic data 3; sparse or judgement-based data 5–10; human error probabilities 5–10.

Where to get data

Use, in order of preference: plant-specific failure history; manufacturer data with a certificate (e.g. IEC 61508 assessment report); recognised generic databases (e.g. OREDA, CCPS process equipment reliability data, SINTEF PDS data handbook, exida SERH). Record the source in the *Notes / data source* field of each event.

8 · Common-cause failures (β-factor)

Redundant channels of the same type often fail together from one cause – same design error, same calibration mistake, same environment, same maintenance team. Ignoring this makes redundancy look far better than it is. The **β-factor model** assumes a fraction β of each channel's failures are common to all channels of the group.

How the app models it: each member event is replaced by `OR( independent part, CCF event )`, where

- independent part: rates (1 − β)·λDU and (1 − βD)·λDD,
- one shared **CCF event** per group: rates β·λDU and βD·λDD, using the geometric mean of the members' rates and the longest member TI.

The CCF event appears in the cut sets as `CCF-xxx` – usually as an order-1 cut set that dominates a redundant subsystem.

Setting up: on the *Common cause (β)* tab click + *Add CCF group* and enter β as a **fraction** (0.05 = 5 %). Leave βD blank to use β. Then on each member event choose the group in *Common-cause group*. (*Build fault tree from this SIF* does this automatically.)

Worked numbers – two valves, λDU = 1×10^{-6} /h, TI = 1 year (λ·TI = 0.00876):

Case	PFDavg
Single valve (1oo1) ≈ λ·TI/2	4.4×10^{-3}
1oo2, β = 0 ≈ (λ·TI)²/3	2.6×10^{-5}

Case	PFDavg
1oo2, β = 10 % ≈ (0.9·λ·TI)²/3 + 0.1·λ·TI/2	4.6×10^{-4} – the common-cause term is 95 % of the result

Typical β (IEC 61508-6 Annex D scoring): logic solvers 0.5–5 %, field sensors and final elements 1–10 %. Justify the value with the Annex D checklist.

9 · How the calculation works

9.1 Exact Boolean calculation (BDD)

The tree is converted into a **Binary Decision Diagram** – a compact, exact representation of the logic. The top-event probability is then calculated exactly, including shared events, voting gates and common-cause events. No rare-event approximation and no independence assumption between branches is needed. Trees with many hundreds of events calculate in milliseconds.

9.2 Time-averaged PFD – why it matters

A proof-tested component's probability of being failed rises from zero just after the test to its maximum just before the next test (a sawtooth). **PFDavg is the time average of the top-event probability**, not a combination of averaged inputs. For redundant (AND) logic the two differ:

```
1oo2, identical channels, no CCF:
correct: PFDavg = (1/T)∫₀ᵀ (λt)² dt = (λT)²/3
averaging inputs first: (λT/2)² = (λT)²/4 → 25 % too low
```

The app therefore evaluates the exact top-event probability at many points across the test cycle and averages them (**"PFDavg – exact, time-averaged (recommended)"**). The value from averaged inputs is shown alongside for comparison only. When proof-test intervals differ (for example transmitters yearly, PLC every 10 years, relief valve every 4 years) the average is taken over the **common repeat cycle** (least common multiple – 20 years in that example) and the number of time points is increased automatically so that every short cycle is resolved. All tests are assumed to be performed at the same time (t = 0, TI, 2TI…). Staggered testing gives a somewhat lower PFDavg for redundant channels, so the simultaneous assumption is conservative.

9.3 Cut sets and approximations

Minimal cut sets are generated bottom-up with absorption (supersets removed). For speed they are truncated by **order** and **probability cut-off** (Settings); the exact BDD result is never truncated. From the cut sets two classic approximations are shown: the **rare-event approximation** Σ Π q (an upper bound for small probabilities) and the **min-cut upper bound** 1 − Π(1 − Π q). They should be close to the exact value when probabilities are small; a large difference indicates high probabilities or heavy truncation.

9.4 Frequencies

If the tree contains an **initiating event frequency**, the result is a frequency. It is calculated exactly as `F = Σⱼ fⱼ × P(top | initiating event j occurs)`, where the conditional probability comes from the exact BDD and is time-averaged over the proof-test cycle like a PFDavg. The cut-set sum `Σ f × Π q` is shown for comparison. Gates that contain an initiating event show their value in /yr; gates below it (for example a protection system) show a probability – their PFDavg. Each cut set must contain exactly one frequency – an AND of two frequencies is dimensionally wrong (a frequency must be combined with probabilities, e.g. “demand AND protection fails on demand”). Typical structure: AND(initiating event, protection layer 1 fails, protection layer 2 fails …).

9.5 Importance and uncertainty

Importance measures (Section 10) use the averaged inputs. The Monte Carlo run samples every event’s value from a lognormal distribution with the event’s mean and error factor (CCF events use the EF of the first group member) and recalculates the time-averaged top event for each sample.

10 · Reading the results

Result	Meaning and use
PFDavg – exact, time-averaged	The value to report for a low-demand safety function. Compare with the target PFD / RRF from the risk assessment (LOPA, risk graph).
Exact, using averaged inputs	Comparison only – shows how much a simple spreadsheet multiplication would differ.
Peak	Highest probability, just before a proof test. Useful to judge how much the test interval matters.
Rare-event / min-cut upper bound	Cut-set approximations; check they are close to the exact value.
RRF and SIL band	RRF = 1 / PFDavg. Low-demand bands: SIL 1 = 10^{-2} to $<10^{-1}$, SIL 2 = 10^{-3} to $<10^{-2}$, SIL 3 = 10^{-4} to $<10^{-3}$, SIL 4 = 10^{-5} to $<10^{-4}$. The band is for PFDavg only (see Section 14).
Top-event frequency	Shown when the tree contains an initiating event; compare with the tolerable frequency.
PFD(t) chart	Top-event probability over the averaging horizon with the average as a dashed line. Steps show where proof tests restore equipment.

Minimal cut sets

Ranked by probability (from averaged inputs) with their share of the total. Look for: **order-1 cut sets** (single points of failure, often CCF events or bypasses); one cut set dominating (improvement should target it); cut sets containing a human action or bypass.

Importance measures

Measure	Definition	Use it to…
Fussell-Vesely (FV)	fraction of the top probability from cut sets containing the event: (P − P(q=0)) / P	find the biggest contributors – where improvement reduces risk most.
Birnbaum	∂P/∂q = P(q=1) − P(q=0)	see how sensitive the top event is to the event's probability.
RAW (risk achievement worth)	P(q=1) / P	judge what happens if the item is out of service or bypassed – high RAW items need tight control of bypasses and repair times.
RRW (risk reduction worth)	P / P(q=0)	the maximum benefit of making the item perfect – a ceiling on what investment in it can achieve.

Uncertainty

Click *Run … samples*. Report the mean and the 5th–95th percentile range. If the 95th percentile falls in a lower SIL band than the point value, the claimed SIL is sensitive to data uncertainty – consider better data, a shorter test interval or more margin.

11 · SIF calculator (IEC 61508-6)

Enter one row per **subsystem** (sensors, logic solver, final elements) with its architecture and **per-channel** data:

- **λD** – total dangerous failure rate of one channel (/h); **DC** – diagnostic coverage (%), giving λDU = λD(1−DC) and λDD = λD·DC;
- **β / βD** – common-cause factors in % for undetected / detected failures (blank βD = β);
- **T1** – proof-test interval (h); **MTTR** – mean time to restore after a detected failure (h); **MRT** – mean repair time after a proof-test-revealed failure (h, blank = MTTR).

The subsystem PFDavg values are summed to give the SIF PFDavg. The equations are shown at the bottom of the tab (IEC 61508-6, B.3.2.2).

Build fault tree from this SIF creates the equivalent tree: one gate per subsystem with the correct failure-voting gate, one proof-tested event per channel and a CCF group per redundant subsystem. Compare the tree's time-averaged PFDavg with the formula result. They normally agree within a few percent. Differences arise because the tree uses the exact exponential sawtooth and exact voting, ignores MRT, and averages over the common test cycle when subsystems have different T1. Once the tree exists you can extend it with things the formulas cannot handle – shared transmitters, bypass probability, different test intervals per channel, additional protection layers.

12 · Worked example – V-101 overpressure protection

Load it with *File ▸ Load example* (or the button at the top of this guide). Data are illustrative.

Top event (AND): overpressure protection of separator V-101 fails on demand = the high-pressure SIF fails **and** relief valve PSV-101 fails to open.

- **SIF (OR)** fails if any of:
  - pressure transmitters PT-101A/B/C fail – **VOTE 2/3** (2oo3 voting), CCF group β = 5 %;
  - safety PLC fails dangerous (10-year proof test);
  - both shut-down valves XV-101/102 fail to close – **AND** (1oo2), CCF group β = 10 %;
  - SIF bypassed (maintenance override left on) – fixed probability 10^{-3}.
- **PSV-101** fails to open – proof-tested every 4 years.

What the results show

- The tree's PFDavg is about **1.6×10^{-4}**, averaged over the 20-year cycle in which all proof tests coincide (1, 4 and 10 years).
- The largest cut sets are **PLC × PSV** (about half of the total), **valve CCF × PSV** and **bypass × PSV**. These show the improvement levers: a shorter PLC proof-test interval, better valve diversity or a lower β, and bypass management.
- The 2oo3 transmitters contribute little: their redundancy works and their CCF term is small.
- The PFD(t) chart shows large steps every 4 years, when the PSV test restores the relief valve. Testing the PSV more often has a large effect.

> Try it: change the PSV TI from 35 040 h (4 yr) to 17 520 h (2 yr) on the *Basic events* tab and watch the PFDavg and the chart change. Then set the bypass probability to 10^{-2} and look at the RAW column.

Example 2 – hot DRI conveyor: nitrogen fire detection & extinguishing (File ▸ Load example: hot DRI conveyor…)

When the enclosed hot DRI conveyor trips or stops under load, hot DRI is left stationary on the belt and can self-heat, re-oxidise and ignite the belt. An automatic N2 system floods the conveyor enclosure. The tree is built as a **frequency** model:

- **Top (AND)** = conveyor trip / stoppage under load (initiating event, 6 /yr) × fire develops on the stationary DRI (0.05) × **N2 system fails on demand (G-002)**.
- **G-002 (OR)**: release not initiated (automatic AND manual initiation both fail) · sufficient N2 not available (low buffer inventory not restored, isolation valve left closed) · N2 not delivered (1oo2 release valves with CCF, pressure regulator, nozzles blocked by DRI fines, 24 V DC solenoid supply) · enclosure not sealed (undeveloped) · system isolated / locked off.
- **Automatic initiation** needs the stoppage-under-load logic (zero-speed switch + belt weigher) *or* fire detection (linear heat cable + 1oo2 CO detectors with CCF) – so both must fail – *or* the extinguishing control panel fails. The panel is a **shared event**: it also processes the N2 low-pressure alarm.

What the results show (illustrative data): fire frequency about 8.6×10^{-3} /yr (once in about 120 years); the N2 system itself has a PFDavg of about 0.03 (RRF ≈ 35). The detection and initiation logic is very reliable because it is diverse and redundant (gate G-003 ≈ 2×10^{-4}). The result is dominated by **single-point failures in the**

delivery path: enclosure not sealed (~35 %), pressure regulator failing closed (~23 %), nozzles blocked by fines (~15 %), isolation valve left closed (~10 %) and system left isolated (~7 %). These are the improvement levers: door / hood interlocks or inspection, a redundant regulator or bypass, regular blow-through testing of the nozzles, and locked-open valves with line-up checks. The Monte Carlo 5–95 % range is wide (about 4×10^{-4} to 3×10^{-2} /yr) because the human-error and conditional probabilities carry large error factors. Replace all data with site and vendor values before using the tree.

13 · Saving, files and export

- **Automatic browser storage** – every change is kept in this browser on this computer (status pill “Saved in this browser”).
- **Linked file** (Chrome / Edge) – *File ▸ Save as…* creates a `.fta.json` file and from then on every change is written to it (“Saved to …”). Put it on OneDrive or a network drive to keep it safe and share it. *File ▸ Open file…* opens one; after restarting the browser you may need to click *Reconnect file* once.
- **Backups** – *Download backup (.json)* / *Import backup* work in every browser.
- **Excel** – *Export results to Excel*: summary, gates, basic events, CCF groups, cut sets, importance, PFD(t) series and the SIF table.
- **Picture** – *Export tree picture (.svg)* for reports and slides (SVG scales without loss; open it in a browser, Word or PowerPoint).
- **Report** – *Print report / PDF*: results, tree drawing, basic event data, top cut sets and importance.

14 · Good practice, assumptions and limitations

Good practice

- Define the top event, system boundary, initial conditions and operating mode before drawing.
- Develop gates by immediate causes; keep descriptions as “fault statements” (*what* fails and *how*).
- Use shared events for genuinely shared equipment; use CCF groups for redundant identical channels.
- Include human and procedural contributions: bypasses, wrong line-up after maintenance, failure to reset.
- Record data sources and assumptions (Notes fields and Settings description); have the tree reviewed by someone who knows the plant.
- Do a sensitivity check on the dominant inputs (test interval, β, bypass probability) and run the uncertainty analysis.

Assumptions built into the calculation

- Constant failure rates (exponential distribution); proof tests are perfect (100 % coverage) and restore the component to “as new”.
- All proof tests are performed simultaneously; test duration and test-induced failures are not modelled.
- λDD contributes λDD × MTTR (the process is not automatically taken to a safe state on detection).
- β-factor CCF model with one CCF event per group.
- Importance measures use averaged inputs.

What the app does not check

- Hardware fault tolerance / safe failure fraction (architectural constraints, IEC 61508-2 Route $1_H/2_H$, IEC 61511-1 clause 11.4) and systematic capability – a SIL claim needs these as well as the PFDavg.
- Spurious trip rate (safe failures), high-demand / continuous-mode PFH.
- Partial stroke testing or imperfect proof-test coverage (model them by splitting λDU into separate events with different test intervals if needed).

> Use the results for analysis, training and design comparison, and confirm SIL verification calculations with your company's verified tool before relying on them.

15 · Troubleshooting and FAQ

The top result is 0 or "–".

The tree has no basic events, a gate has no inputs (treated as FALSE – see the ⚠ notes), the top event depends on a FALSE house event, or the tree contains a frequency (then the result is shown as a frequency).

"Cut-set list truncated".

Only the listed cut sets are affected. Raise the order limit or lower the cut-off in Settings if you need the full list; the exact PFDavg is unaffected.

The rare-event value is much larger than the exact value.

Some probabilities are not small (≥ 0.1). Use the exact value.

Why is the time-averaged PFDavg higher than the "averaged inputs" value?

That is expected for redundant (AND / voting) logic – see Section 9.2. The time-averaged value is the correct one.

"Proof-test intervals differ – averaged over their common cycle…".

Information only: the app averaged over the full repeating cycle. If intervals have no practical common cycle (e.g. 8 760 h and 10 000 h) it averages over the longest interval and says the result is approximate – use rounded, commensurate intervals if possible.

"… cut sets do not contain exactly one initiating-event frequency".

A frequency event is ANDed with another frequency, or an OR branch has no frequency. Restructure so each path = one frequency AND conditional probabilities.

An edit applied to several places.

The event is shared (↺). Use *Make independent copy here* to separate it.

"Reconnect file" keeps appearing.

The browser asks for permission to write the linked file again after a restart – click it once. If the file was moved or renamed, use *Save as…* again.

Can two people work on the same file?

Not at the same time – the last save wins. Keep one master file and exchange backups otherwise.

16 · Glossary

BDD	Binary Decision Diagram – exact, compact representation of the tree logic used for the calculation.
β, βD	Common-cause factors: fraction of undetected (β) or detected (βD) dangerous failures that affect all channels of a group.
CCF	Common-cause failure – simultaneous failure of several items from one shared cause.
Cut set / minimal cut set	Combination of basic events that causes the top event / one that cannot be reduced.
DC	Diagnostic coverage – fraction of dangerous failures detected by diagnostics.
EF	Error factor – 95th / 50th percentile ratio of a lognormal uncertainty distribution.
IPL	Independent protection layer (LOPA term) – each can be one branch of an AND top event.
λD, λDU, λDD	Dangerous failure rate; dangerous undetected; dangerous detected (per hour).
MooN	M-out-of-N voting: M of N channels must act to perform the safety function.
MTTR / MRT	Mean time to restore after a detected failure / mean repair time after a failure found by proof test.
PFD, PFDavg	Probability of failure on demand; its average over time.
Proof test, TI / T1	Periodic test that reveals undetected dangerous failures; its interval.
RRF	Risk reduction factor = 1 / PFDavg.
SIF / SIS / SIL	Safety instrumented function / system; safety integrity level (1–4).
Top event	The undesired event the fault tree analyses.

17 · References

- IEC 61508-6:2010 – Functional safety of E/E/PE safety-related systems, Part 6: Guidelines on the application of IEC 61508-2 and -3 (Annex B: PFD equations; Annex D: β-factor estimation).
- IEC 61511-1:2016 – Functional safety: Safety instrumented systems for the process industry sector.
- ISA-TR84.00.02 – Safety Integrity Level (SIL) verification of safety instrumented functions.
- NUREG-0492 (1981) – Fault Tree Handbook, U.S. Nuclear Regulatory Commission.
- NASA (2002) – Fault Tree Handbook with Aerospace Applications, version 1.1.
- CCPS – Guidelines for Hazard Evaluation Procedures (fault tree analysis chapter); Layer of Protection Analysis: Simplified Process Risk Assessment.
- A. Rauzy (1993) – New algorithms for fault trees analysis, *Reliability Engineering & System Safety* 40 (BDD method).